

Ital Pascal

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET	V2.0.6	DOWNLOADS	1.1K	LICENSE	MIT	GITHUB PAGES	ONLINE
-------	--------	-----------	------	---------	-----	--------------	--------

◆ Analisi della classe **DB** (VB.NET)

1. Scopo generale del codice

La classe **DB** è un *utility* statica (non istanziabile) che fornisce un set di metodi di accesso ai dati per un'applicazione .NET che utilizza **SQL Server**.

Il suo obiettivo è:

- **Centralizzare** la gestione delle connessioni e delle query.
- Offrire **metodi di alto livello** (scalar, non-query, reader, DataTable, DataSet) per eseguire SQL in modo semplice e sicuro.
- Fornire **funzioni di supporto** per la creazione dinamica di parametri e per le query "LIKE" con più parametri.

In pratica, è un "wrapper" intorno a `System.Data.SqlClient` che riduce il boilerplate di apertura/chiusura connessioni e di gestione dei parametri.

2. ⓘ Componenti chiave e funzioni

Sezione	Funzione	Descrizione
Connessione	<code>GetConnection(Optional name As String = "MiaConnessione")</code>	Recupera la stringa di connessione dal file di configurazione (<code>App.config / Web.config</code>) e restituisce un oggetto <code>SqlConnection</code> . Lancia un'eccezione se la stringa non esiste.

Sezione	Funzione	Descrizione
Scalar	<code>ExecuteScalar(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")</code>	Esegue una query che restituisce un singolo valore (es. <code>COUNT</code> , <code>MAX</code>). Restituisce l'oggetto risultante.
Non-Query	<code>ExecuteNonQuery(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")</code>	Esegue <code>INSERT</code> , <code>UPDATE</code> o <code>DELETE</code> . Restituisce il numero di righe interessate. Se il risultato è <code>0</code> , scrive un avviso su <code>Debug</code> .
Reader	<code>ExecuteReader(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")</code>	Restituisce un <code>SqlDataReader</code> con <code>CommandBehavior.CloseConnection</code> per chiudere automaticamente la connessione al termine della lettura.
DataTable	<code>FillDataTable(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")</code>	Riempie e restituisce un <code>DataTable</code> (utile per visualizzazioni in UI come <code>DataGridView</code>).
DataSet	<code>FillDataSet(query As String, Optional parameters As List(Of SqlParameter) = Nothing, Optional connectionName As String = "MiaConnessione")</code>	Simile a <code>FillDataTable</code> ma per più tabelle (riempie un <code>DataSet</code>).
Utility Parametri	<code>AddParameters(cmd As SqlCommand, parameters As List(Of SqlParameter))</code>	Aggiunge un array di parametri a un <code>SqlCommand</code> .
Query semplificata	<code>Query(sql As String, ParamArray values() As Object)</code>	Crea automaticamente parametri <code>@p1</code> , <code>@p2</code> , ... a partire dagli argomenti passati e restituisce un <code>DataTable</code> .
Query LIKE	<code>QueryLike(sql As String, ParamArray values() As String)</code>	Avvolge <code>Query</code> aggiungendo i caratteri <code>%</code> ai valori per le clausole <code>LIKE</code> .
Metadati	<code>GetTables()</code>	Restituisce un <code>DataTable</code> con i nomi delle tabelle di base presenti nel database.

1. **Chiamata di un metodo** (es. `ExecuteNonQuery`).

2. Connessione

- `GetConnection` recupera la stringa di connessione dal file di configurazione.
- Viene creato un oggetto `SqlConnection` e aperto (`conn.Open()`).

3. Comando

- Viene istanziato un `SqlCommand` con la query e la connessione.
- Se sono presenti parametri, `AddParameters` li aggiunge al comando.

4. Esecuzione

- `ExecuteNonQuery` , `ExecuteScalar` , `ExecuteReader` , `SqlDataAdapter.Fill` , ecc. vengono chiamati in base al metodo richiesto.

5. Pulizia

- I blocchi `Using` garantiscono la chiusura e il rilascio delle risorse (`conn` , `cmd` , `da`).
- Per `ExecuteReader` , la chiusura avviene automaticamente grazie a `CommandBehavior.CloseConnection` .

4. Punti di forza

Punti di forza

- **Semplicità:** i metodi richiedono solo la query e, opzionalmente, i parametri.
- **Sicurezza:** l'uso di `SqlParameter` previene l'injection SQL.
- **Riutilizzabilità:** tutti i metodi sono statici, quindi non è necessario creare istanze.
- **Gestione automatica delle risorse:** i blocchi `Using` evitano perdite di connessioni.

5. Esempio di utilizzo

```
1  ' Inserimento di un nuovo cliente
2  Dim sqlInsert = "INSERT INTO Clienti (Nome, Città) VALUES (@p1, @p2)"
3  Dim rows = DB.ExecuteNonQuery(sqlInsert, New List(Of SqlParameter) From
4      {
5          New SqlParameter("@p1", "Mario"),
6          New SqlParameter("@p2", "Roma")
7      })
8
9  ' Recupero di tutti i clienti
10 Dim dt = DB.FillDataTable("SELECT * FROM Clienti")
```

```
10
11 ' Ricerca con LIKE
12 Dim dtLike = DB.QueryLike("SELECT * FROM Clienti WHERE Citta LIKE @p1",
    "Roma")
```

Conclusione

La classe `DB` è un helper di accesso ai dati che semplifica notevolmente la scrittura di codice di persistenza in VB.NET. Fornisce un'interfaccia coerente e sicura per eseguire query, gestire parametri e ottenere risultati in vari formati, riducendo il rischio di errori e migliorando la manutenibilità del progetto.